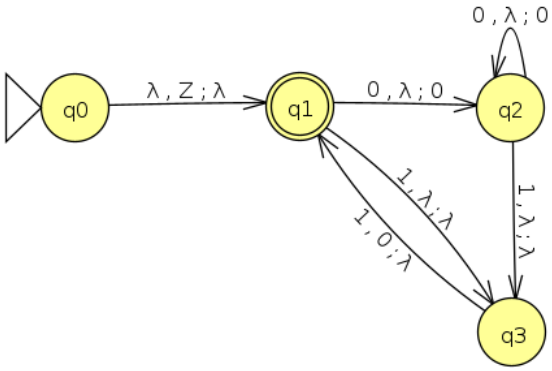
Práctica 3 TALF

José Ruiz Pareja

6 de mayo de 2026

1. Autómata con pila no determinista:

Construye un APND en JFLAP que reconozca el lenguaje L = $0^n 1^{2n} : n > 0$



Verifica el autómata construido con 6 cadenas de ejemplo.
Cadenas de ejemplo:

Input	Result
	Accept
01	Reject
001111	Accept
011	Accept
000111111	Accept
00111	Reject

Haz lo mismo en Python, definiendo el autómata en el JSON e instanciándolo con los mismos ejemplos.
APND definido en el JSON:

```
{
"name" : "APND1",
"hint" : "",
"e.g." : [],
"representation" : {
  "K" : ["q0", "q1"],
  "I" : ["0", "1"],
  "S" : ["0", "1"],
  "s" : "q0",
  "F" : ["q0"],
  "t" : [[["q0", "0", "ε"],["q1", "0"]],
         [["q0",  "1", "ε"],["q2", "ε"]],
         [["q1",  "0", "ε"],["q1", "0"]],
```

```
            [["q1",  "1", "ε"],["q2", "ε"]],
            [["q2",  "1", "0"],["q0", "ε"]]]
    }
}
```

Y probamos con los mismos ejemplos:

```
>>> from python.automata import cfg_to_npa, pushdown_automaton
>>> pushdown_automaton("APND1", "")

M = ({q0, q1}, {0, 1}, {0, 1}, {((q0, 0, ε), (q1, 0)), ((q0, 1, ε), (q2, ε)),
((q1, 0, ε), (q1, 0)), ((q1, 1, ε), (q2, ε)), ((q2, 1, 0), (q0, ε))}, q0, {q0}
)

w =

(q0, ε, ε)

w ∈ 𝒞(M)
'accept'
>>> pushdown_automaton("APND1", "01")

M = ({q0, q1}, {0, 1}, {0, 1}, {((q0, 0, ε), (q1, 0)), ((q0, 1, ε), (q2, ε)),
((q1, 0, ε), (q1, 0)), ((q1, 1, ε), (q2, ε)), ((q2, 1, 0), (q0, ε))}, q0, {q0}
)

w = 01

(q0, 01, ε) ⊢ (q1, 1, 0) ⊢ (q2, ε, 0)

Blocked computation, w not accepted by M.
'blocked'
>>> pushdown_automaton("APND1", "001111")

M = ({q0, q1}, {0, 1}, {0, 1}, {((q0, 0, ε), (q1, 0)), ((q0, 1, ε), (q2, ε)),
((q1, 0, ε), (q1, 0)), ((q1, 1, ε), (q2, ε)), ((q2, 1, 0), (q0, ε))}, q0, {q0}
)

w = 001111

(q0, 001111, ε) ⊢ (q1, 01111, 0) ⊢ (q1, 1111, 00) ⊢ (q2, 111, 00) ⊢ (q0, 11, 0
) ⊢ (q2, 1, 0) ⊢ (q0, ε, ε)

w ∈ 𝒞(M)
'accept'
>>> pushdown_automaton("APND1", "011")

M = ({q0, q1}, {0, 1}, {0, 1}, {((q0, 0, ε), (q1, 0)), ((q0, 1, ε), (q2, ε)),
((q1, 0, ε), (q1, 0)), ((q1, 1, ε), (q2, ε)), ((q2, 1, 0), (q0, ε))}, q0, {q0}
)

w = 011

(q0, 011, ε) ⊢ (q1, 11, 0) ⊢ (q2, 1, 0) ⊢ (q0, ε, ε)

w ∈ 𝒞(M)
'accept'
>>> pushdown_automaton("APND1", "000111111")

M = ({q0, q1}, {0, 1}, {0, 1}, {((q0, 0, ε), (q1, 0)), ((q0, 1, ε), (q2, ε)),
((q1, 0, ε), (q1, 0)), ((q1, 1, ε), (q2, ε)), ((q2, 1, 0), (q0, ε))}, q0, {q0}
)

w = 000111111

(q0, 000111111, ε) ⊢ (q1, 00111111, 0) ⊢ (q1, 0111111, 00) ⊢ (q1, 111111, 000)
 ⊢ (q2, 11111, 000) ⊢ (q0, 1111, 00) ⊢ (q2, 111, 00) ⊢ (q0, 11, 0) ⊢ (q2, 1, 0
) ⊢ (q0, ε, ε)

w ∈ 𝒞(M)
'accept'
>>> pushdown_automaton("APND1", "00111")

M = ({q0, q1}, {0, 1}, {0, 1}, {((q0, 0, ε), (q1, 0)), ((q0, 1, ε), (q2, ε)),
((q1, 0, ε), (q1, 0)), ((q1, 1, ε), (q2, ε)), ((q2, 1, 0), (q0, ε))}, q0, {q0}
)

w = 00111

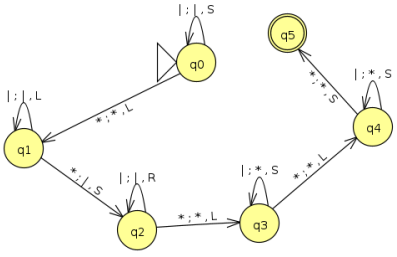
(q0, 00111, ε) ⊢ (q1, 0111, 0) ⊢ (q1, 111, 00) ⊢ (q2, 11, 00) ⊢ (q0, 1, 0) ⊢ (
q2, ε, 0)

Blocked computation, w not accepted by M.
'blocked'
```

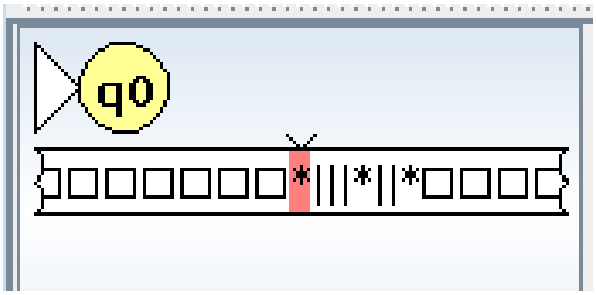
2. Máquina de Turing:

Define en JFLAP la MT que define la suma de dos números naturales.
Si copiamos la MT de la solucion del ejercico 12 tal cual, quedaría tal que asi:

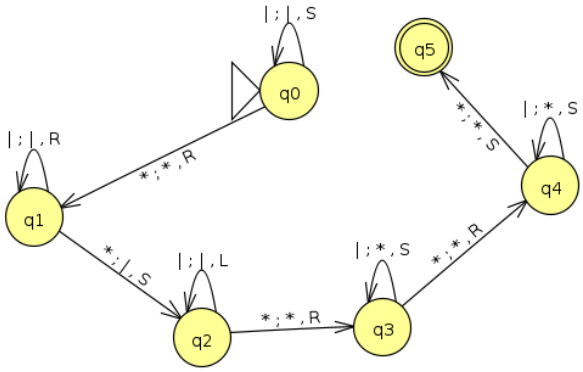
0	*	l	1
0	|	|	0
1	*	|	2
1	|	l	1
2	*	l	3
2	|	r	2
3	*	l	4
3	|	*	3
4	*	h	4
4	|	*	4



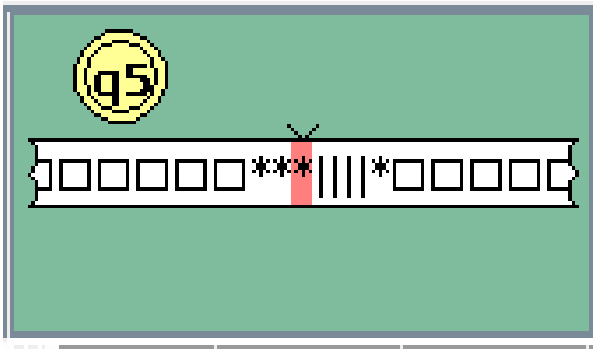
Sin embargo, en JFLAP la máquina de Turing se posiciona al inicio delante de la cadena



Por lo que, para que la máquina funcione, se deberian de invertir todos los desplazamientos de la máquina (izquierda por derecha y viceversa). La MT en JFLAP tendría el siguiente aspecto con los desplazamientos ya invertidos:



De este modo, ya se puede realizar la funcion suma (2+1):



Comprueba su correcto funcionamiento comparándolo con su ejecución en Python.

En python, comprobamos que con la misma entrada ”$*|||*||*$”, funciona y además no hace falta invertir los desplazamientos.



3. Función recursiva

Sabemos que $suma(n_1 + n_2) = n_1 + n_2$ es una funcion recursiva, ya que se puede definir como: $suma(n) = \{\pi^1_1 | sucesor_3\}(n)$

Por lo que la suma de 3 numeros se podría definir como:
$suma(suma(n_1 + n_2) + n_3) = suma(n_1 + n_2) + n_3 = n_1 + n_2 + n_3$
La suma de 3, 2 y 1 quedaría como: $suma(suma(1 + 2) + 3) = suma(1 + 2) + 3 = 3 + 3 = 6$
Compara tu evaluación con su ejecución en Python.

```
>>> from python.recursivefunctions import *
>>> eval_rec_function('addition', (eval_rec_function('addition', 3, 1)),2)
addition(3,1)
<π¹₁|σ(π³₃)>(3,1)
<π¹₁|σ(π³₃)>(3,0)
π¹₁(3) = 3
σ(π³₃)(3,0,3)
π³₃(3,0,3) = 3

σ(3) = 4
addition(4,2)
<π¹₁|σ(π³₃)>(4,2)
<π¹₁|σ(π³₃)>(4,1)
<π¹₁|σ(π³₃)>(4,0)
π¹₁(4) = 4
σ(π³₃)(4,0,4)
π³₃(4,0,4) = 4

σ(4) = 5
σ(π³₃)(4,1,5)
π³₃(4,1,5) = 5

σ(5) = 6
6
```

Como podemos ver, en python tambien se resuelve por partes la funcion recursiva.